Cryptanalysis of a Type of White-Box Implementations of the SM4 Block Cipher

Jiqiang Lu[1,2,3], Jingyu Li[1], Zexuan Chen[1], and Yanan Li[1]

[1] *School of Cyber Science and Technology, Beihang University, Beijing 100083, China*
[2] *Guangxi Key Laboratory of Cryptography and Information Security, Guilin 541004, China*
[3] *Hangzhou Innovation Institute, Beihang University, Hangzhou 310053, China*
Email: {lvjiqiang, lijingyu98, czxyeah, leeyn}@buaa.edu.cn

The SM4 block cipher is a Chinese national standard and an ISO international standard. Since white-box cryptography has many real-life applications nowadays, a few white-box implementations of SM4 has been proposed, among which a type of constructions is dominated, that uses a linear or affine diagonal block encoding to protect the original three 32-bit branches entering a round function and uses its inverse as the input encoding to the S-box layer. In this paper, we analyse the security of this type of constructions against Lepoint et al.'s collision-based attack method, our experiment under a small fraction of (encodings, round key) combinations shows that the rank of the concerned linear system is much less than the number of the involved unknowns, meaning these white-box SM4 implementations should resist Lepoint et al.'s method, but we leave it as an open problem whether there are such encodings that the rank of the corresponding linear system is slightly less than the number of the involved unknowns, in which scenario Lepoint et al.'s method may be used to recover a round key for the case with linear encodings and to remove most white-box operations until mainly some Boolean masks for the case with affine encodings.

Keywords: Cryptology; White-Box Cryptography; SM4 Block Cipher; Collision Attack

1. INTRODUCTION

In 2002, Chow et al. [1] introduced white-box cryptography and proposed a white-box implementation of the AES [2] block cipher. White-box cryptography works under the white-box security model, which assumes an attacker has full access to the execution environment and execution details of a cryptographic implementation, giving the attacker more power than the black-box and grey-box security models. Nowadays, white-box cryptography has many real-life application scenarios, and some white-box cryptography solutions have been in use.

The primary security threat for white-box cryptography is key extraction attack, which aims to extract the key used in a white-box implementation. Chow et al.'s white-box AES implementation has been cryptanalysed extensively [3–5], particularly, in 2004 Billet et al. [3] presented an algebraic attack with a time complexity of 2^{30}, and in 2013 Lepoint et al. [5] improved Billet et al.'s attack to have a time complexity of 2^{22} and presented a collision-based attack with a time complexity of 2^{22}. Besides, a few different white-box AES implementations have been proposed [6–11], but all of them have been broken with a practical or semi-practical time complexity [5, 11–15]. Generally speaking, it has been well understood that the line of white-box implementation for an existing cryptographic algorithm is hardly possible to achieve the full security as in the black-box model, but it is expected that it can provide some protection with a realistic significance.

The SM4 block cipher was first released in 2006 as the SMS4 [16] block cipher used in the Chinese national standard WAPI (WLAN Authentication and Privacy Infrastructure), which has a generalised Feistel structure with a 128-bit block length, a 128-bit user key and a total of 32 rounds. SMS4 became a Chinese cryptographic industry standard in 2012, labeled with SM4, which then became a Chinese national standard [17] in 2016 and an ISO international standard [18] in 2021. The main white-box implementation results of SMS4/SM4 are as follows. In 2009, Xiao and Lai [19] proposed a white-box SM4 implementation with lookup tables and affine transformations. In 2013, Lin and Lai [20] attacked Xiao and Lai's white-box SM4 implementation with a time complexity of around 2^{47} by combining Billet et al.'s attack with other techniques like

differential cryptanalysis [21]. In 2015, Shi et al. [22] proposed a lightweight white-box SM4 implementation based on the idea of dual cipher [23]. In 2016, Shang [24] improved Xiao and Lai's white-box SM4 implementation mainly by building a white-box table with two S-boxes, and got a security level of around 2^{48}; and Bai and Wu [25] proposed a white-box SM4 implementation with an S-box input being divided into two shares. In 2018, Pan et al. [26] showed that the time complexity of Lin and Lai's attack on Xiao and Lai's white-box SM4 implementation is overestimated and the security of either Xiao and Lai's or Bai and Wu's white-box SM4 implementation is mainly based on the constants of the affine external encodings; and Lin et al. [27] applied Biryukov et al.'s affine equivalence algorithm [28] to attack Shi et al.'s white-box SM4 implementation with a time complexity of 2^{49}. In 2020, Yao and Chen [29] proposed a white-box SM4 implementation with some original internal states expanded by dummy states under the control of a secret random number, and got the lowest attack complexity of about 2^{51} among a few attack techniques. In 2022, Wang et al. [30, 31] applied Lepoint et al.'s collision-based idea to attack Shi et al.'s white-box SM4 implementation with a time complexity of about 2^{23}.

In this paper, we are concerned with Xiao and Lai's, Shi et al.'s, Shang's and Yao and Chen's white-box SM4 implementations [19, 22, 24, 29], which are more or less different one another from a structural view but fundamentally all employ the construction method that uses a linear or affine diagonal block encoding to protect the original three branches entering a round function and uses its inverse as the input encoding to the S-box layer. We analyse the security of this type of white-box SM4 implementations against Lepoint et al.'s collision-based attack method, assuming that the attacker know the design details of these implementations. Our small-scale experiments under a few (encodings, round key) combinations show that the rank of the concerned linear system is much less than the number of the involved unknowns, which means that it is infeasible to apply Lepoint et al.'s collision-based attack to the four implementations, but on the other hand, it is impossible to experimentally test all encodings, and at present we are not sure whether there are encodings such that the rank of the corresponding linear system is slightly less than the number of the involved unknowns, and if this case is supposed to exist we show that Lepoint et al.'s collision-based attack method could be applied with a practical time complexity to remove most white-box operations until mainly some Boolean encodings remaining for Yao et al.'s, Xiao and Lai's and Shang's implementations that use affine encodings, and to recover a round key for Shi et al.'s implementation that uses linear encodings. Our cryptanalysis suggests to some extent that for white-box SM4 implementation, building a white-box table with one or two S-boxes is both okay and affine encodings are preferable to linear encodings in the sense of their security against Lepoint et al.'s collision-based attack method, but nevertheless we leave it as an open problem whether there are such encodings that the rank of the corresponding linear system is slightly less than the number of the involved unknowns.

The remainder of the paper is organised as follows. We describe the notation and the SM4 block cipher in the next section, and present our cryptanalysis on Yao and Chen's, Xiao and Lai's, Shi et al.'s and Shang's white-box SM4 implementations in Sections 3 to 6, respectively. Section 7 concludes this paper.

2. PRELIMINARIES

In this section, we give the notation used throughout this paper, and briefly describe the SM4 block cipher.

2.1. Notation

In all descriptions we assume that a number without a prefix is in decimal notation, and a number with prefix $0x$ is in hexadecimal notation. We use the following notation throughout this paper.

$\oplus$	bitwise exclusive OR (XOR)
$\gg$	right shift of a bit string
$\lll$	left rotation of a bit string
$\|\|$	bit string concatenation
$\circ$	functional composition

2.2. The SM4 Block Cipher

SM4 [16, 17] is a generalised Feistel cipher with 32 rounds, a 128-bit block size and a 128-bit key length. Denote by $(X_i, X_{i+1}, X_{i+2}, X_{i+3})$ the 128-bit input to the i-th round, by rk_i the 32-bit i-th round key, where $X_i \in \mathrm{GF}(2)^{32}$ and $i = 0, 1, \ldots, 31$.

Define the nonlinear function $\tau : \mathrm{GF}(2)^{32} \to \mathrm{GF}(2)^{32}$ that applies the same 8-bit S-box $\mathbf{S}$ four times in parallel as

$$x \mapsto \big(\mathbf{S}(x_{[31\ldots24]}), \mathbf{S}(x_{[23\ldots16]}), \mathbf{S}(x_{[15\ldots8]}), \mathbf{S}(x_{[7\ldots0]})\big);$$

and define the linear function $\mathbf{L} : \mathrm{GF}(2)^{32} \to \mathrm{GF}(2)^{32}$ as

$$x \mapsto x \oplus (x \lll 2) \oplus (x \lll 10) \oplus (x \lll 18) \oplus (x \lll 24). \quad (1)$$

Then, the invertible transformation $\mathbf{T} : \mathrm{GF}(2)^{32} \times \mathrm{GF}(2)^{32} \to \mathrm{GF}(2)^{32}$ is defined to be

$$(x, rk_i) \to \mathbf{L}(\tau(x \oplus rk_i)),$$

and the round function $\mathbf{F} : \mathrm{GF}(2)^{128} \times \mathrm{GF}(2)^{32} \to \mathrm{GF}(2)^{128}$ under round key rk_i is

$$((X_i, X_{i+1}, X_{i+2}, X_{i+3}), rk_i) \mapsto (X_{i+1}, X_{i+2}, X_{i+3}, X_i \oplus \mathbf{T}\big(X_{i+1} \oplus X_{i+2} \oplus X_{i+3}, rk_i\big)\big). \quad (2)$$

The encryption procedure of SM4, as depicted in Fig. 1, consists of the 32 round functions $\mathbf{F}$'s and finally

a reverse transformation $R : \mathrm{GF}(2)^{128} \to \mathrm{GF}(2)^{128}$ defined as

$$(X_{32}, X_{33}, X_{34}, X_{35}) \mapsto (X_{35}, X_{34}, X_{33}, X_{32}).$$

The decryption process of SM4 is the same as the encryption process, except that the round keys are used in the reverse order. We refer the reader to [16, 17] for detailed specifications.

Particularly, as in [29], it is easy that the linear transformation $\mathbf{L}$ (as described in Eq. (1)) of SM4 can also be represented as an invertible 32×32-bit matrix

$$\begin{bmatrix} B_1 & B_2 & B_2 & B_3 \\ B_3 & B_1 & B_2 & B_2 \\ B_2 & B_3 & B_1 & B_2 \\ B_2 & B_2 & B_3 & B_1 \end{bmatrix}, \tag{3}$$

with B_1, B_2 and B_3 being invertible 8×8-bit matrices.

Let x_0, x_1, x_2, x_3 be four byte variables, represent $\mathbf{L}$ as four 32×8-bit matrices $\begin{bmatrix} \mathbf{L}_0 & \mathbf{L}_1 & \mathbf{L}_2 & \mathbf{L}_3 \end{bmatrix}$, and define

$$\begin{aligned} \mathbf{L}_0(x) &= x \cdot \begin{bmatrix} B_1 & B_3 & B_2 & B_2 \end{bmatrix}^T, \\ \mathbf{L}_1(x) &= x \cdot \begin{bmatrix} B_2 & B_1 & B_3 & B_2 \end{bmatrix}^T, \\ \mathbf{L}_2(x) &= x \cdot \begin{bmatrix} B_2 & B_2 & B_1 & B_3 \end{bmatrix}^T, \\ \mathbf{L}_3(x) &= x \cdot \begin{bmatrix} B_3 & B_2 & B_2 & B_1 \end{bmatrix}^T, \end{aligned}$$

then $\mathbf{L}(x_0||x_1||x_2||x_3) = \mathbf{L}_0(x_0) \oplus \mathbf{L}_1(x_1) \oplus \mathbf{L}_2(x_2) \oplus \mathbf{L}_3(x_3)$.

3. SECURITY OF YAO AND CHEN'S WHITE-BOX SM4 IMPLEMENTATION AGAINST COLLISION-BASED ATTACK

In this section, we describe Yao and Chen's white-box SM4 implementation and analyse its security against Lepoint et al.'s collision-based attack method.

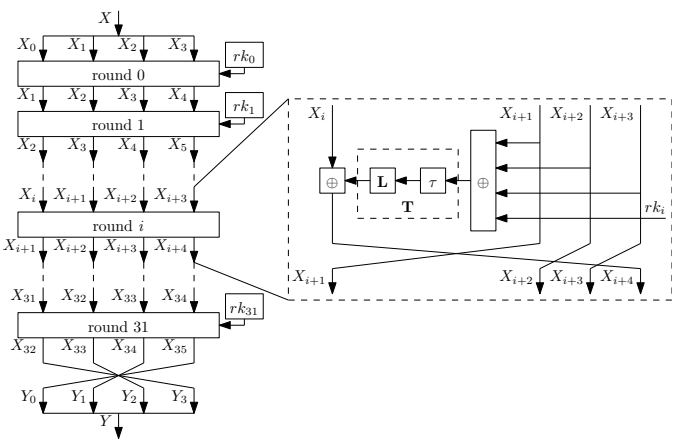

FIGURE 1. SM4 encryption procedure

3.1. Yao and Chen's White-Box SM4 Implementation

Yao and Chen's white-box SM4 implementation [29] is based on internal state expansion, particularly, the 32×32-bit matrix representation Eq. (3) of the linear transformation $\mathbf{L}$ is expanded to the following 64×64-bit matrix $\widehat{\mathbf{L}}$ with the 8×8-bit zero matrix $\mathbf{0}$:

$$\widehat{\mathbf{L}} = \begin{bmatrix} B_1 & \mathbf{0} & B_2 & \mathbf{0} & B_2 & \mathbf{0} & B_3 & \mathbf{0} \\ \mathbf{0} & B_1 & \mathbf{0} & B_2 & \mathbf{0} & B_2 & \mathbf{0} & B_3 \\ B_3 & \mathbf{0} & B_1 & \mathbf{0} & B_2 & \mathbf{0} & B_2 & \mathbf{0} \\ \mathbf{0} & B_3 & \mathbf{0} & B_1 & \mathbf{0} & B_2 & \mathbf{0} & B_2 \\ B_2 & \mathbf{0} & B_3 & \mathbf{0} & B_1 & \mathbf{0} & B_2 & \mathbf{0} \\ \mathbf{0} & B_2 & \mathbf{0} & B_3 & \mathbf{0} & B_1 & \mathbf{0} & B_2 \\ B_2 & \mathbf{0} & B_2 & \mathbf{0} & B_3 & \mathbf{0} & B_1 & \mathbf{0} \\ \mathbf{0} & B_2 & \mathbf{0} & B_2 & \mathbf{0} & B_3 & \mathbf{0} & B_1 \end{bmatrix}.$$

Represent the matrix $\widehat{\mathbf{L}}$ as four 64×16-bit matrices, that is, $\widehat{\mathbf{L}} = \begin{bmatrix} \widehat{\mathbf{L}}_0 & \widehat{\mathbf{L}}_1 & \widehat{\mathbf{L}}_2 & \widehat{\mathbf{L}}_3 \end{bmatrix}$. Then, an encryption round of Yao and Chen's implementation consists of the following three parts according to Eq. (2), as depicted in Fig. 2. Note first that X_l is the corresponding original value protected with an affine output encoding $P_l(x) = A_l \cdot x \oplus a_l$, where x is a 32-bit variable, the linear part A_l is a secret (randomly generated) general invertible 32×32-bit matrix, the constant part a_l is a secret (randomly generated) 32-bit vector, and $l = 0, 1, \cdots, 35$.

3.1.1. Part 1 – Implement $X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \mapsto Z_i$
In order to obtain the original value of $X_{i+1} \oplus X_{i+2} \oplus X_{i+3}$ from the protected forms X_{i+1}, X_{i+2} and X_{i+3}, apply first the inverses P_{i+1}^{-1}, P_{i+2}^{-1} and P_{i+3}^{-1} of the three output encodings respectively to X_{i+1}, X_{i+2} and X_{i+3}, followed by an identical diagonal output encoding $E_i = \mathrm{diag}(E_{i,0}, E_{i,1}, E_{i,2}, E_{i,3})$, where $E_{i,0}, E_{i,1}, E_{i,2}, E_{i,3}$ are four general invertible 8×8-bit affine transformations $(i = 0, 1, \cdots, 31)$. We write $E_{i,l}(\cdot) = C_{i,l}(\cdot) \oplus c_{i,l}$, where the linear part $C_{i,l}$ is a general invertible 8×8-bit matrix and $c_{i,l}$ is an 8-bit constant $(l = 0, 1, 2, 3)$.

This part can be summarised as

$$\begin{aligned} X'_{i+j} &= E_i \circ P_{i+j}^{-1}(X_{i+j}), \quad j = 1, 2, 3; \\ Z_i &= X'_{i+1} \oplus X'_{i+2} \oplus X'_{i+3}, \end{aligned}$$

where Z_i is a 32-bit variable. Observe that the final result of this part $Z_i = E_i \circ (P_{i+1}^{-1}(X_{i+1}) \oplus P_{i+2}^{-1}(X_{i+2}) \oplus P_{i+3}^{-1}(X_{i+3}))$ is the original value of $X_{i+1} \oplus X_{i+2} \oplus X_{i+3}$ protected with the output encoding E_i in such a way that its four bytes are protected respectively with the four 8-bit encodings $E_{i,0}, E_{i,1}, E_{i,2}$ and $E_{i,3}$.

3.1.2. Part 2 – Implement $\mathbf{T}(Z_i, rk_i) \mapsto Y_i (= Y_{i0}||Y_{i1})$
The input Z_i of the second part is the output of the first part, represent Z_i as 4 bytes $Z_i = (z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3})$, and represent the round key rk_i as 4 bytes $rk_i = (rk_{i,0}, rk_{i,1}, rk_{i,2}, rk_{i,3})$, where $i = 0, 1, \cdots, 31$. Next,

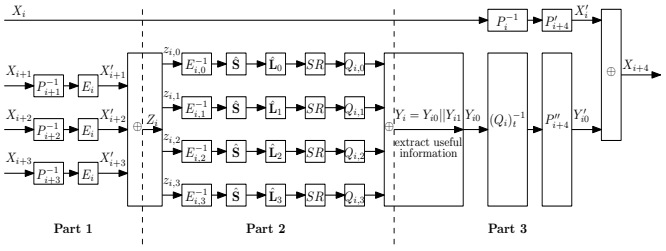

FIGURE 2. An encryption round of Yao and Chen's white-box SM4 implementation

construct four lookup tables that map from 8-bit input to 64-bit output each, as follow:

$$Table_{i,0} = G_{i,0} \circ \widehat{\mathbf{L}}_0[\widehat{\mathbf{S}}(E_{i,0}^{-1}(z_{i,0}), rk_{i,0}, \alpha_{i,0})_{t_{i,0}}],$$
$$Table_{i,1} = G_{i,1} \circ \widehat{\mathbf{L}}_1[\widehat{\mathbf{S}}(E_{i,1}^{-1}(z_{i,1}), rk_{i,1}, \alpha_{i,1})_{t_{i,1}}],$$
$$Table_{i,2} = G_{i,2} \circ \widehat{\mathbf{L}}_2[\widehat{\mathbf{S}}(E_{i,2}^{-1}(z_{i,2}), rk_{i,2}, \alpha_{i,2})_{t_{i,2}}],$$
$$Table_{i,3} = G_{i,3} \circ \widehat{\mathbf{L}}_3[\widehat{\mathbf{S}}(E_{i,3}^{-1}(z_{i,3}), rk_{i,3}, \alpha_{i,3})_{t_{i,3}}],$$

where

- $\alpha_{i,j}$ is an 8-bit random number ($j = 0, 1, 2, 3$);
- $\widehat{\mathbf{L}}_j$ is the corresponding j-th 64×16-bit part of $\widehat{\mathbf{L}}$;
- $(t_{i,0}, t_{i,1}, t_{i,2}, t_{i,3})$ ($t_{i,j} \in \{0, 1\}$) is a 4-bit random vector, and

$$\widehat{\mathbf{S}}(E_{i,j}^{-1}(z_{i,j}), rk_{i,j}, \alpha_{i,j})_{t_{i,j}} = \begin{cases} \mathbf{S}(E_{i,j}^{-1}(z_{i,j}) \oplus rk_{i,j}) \parallel \mathbf{S}(E_{i,j}^{-1}(z_{i,j}) \oplus \alpha_{i,j}), & t_{i,j} = 0; \\ \mathbf{S}(E_{i,j}^{-1}(z_{i,j}) \oplus \alpha_{i,j}) \parallel \mathbf{S}(E_{i,j}^{-1}(z_{i,j}) \oplus rk_{i,j}), & t_{i,j} = 1. \end{cases}$$

  That is, the $\widehat{\mathbf{S}}$ operation is constructed by expanding the original $\mathbf{S}$ operation with a dummy $\mathbf{S}$ operation under the control of the 1-bit $t_{i,j}$ parameter.
- $G_{i,j}$ is the composition of a shift matrix SR and an output encoding $Q_{i,j}$. The shift matrix SR transforms the expanded 64-bit value after $\widehat{\mathbf{L}}_j$ into such a 64-bit value that the former half is the original 32-bit part (without expansion) and the latter half consists only of some dummy bits. $Q_{i,j}$ is of the affine form $Q_{i,j}(x) = L_Q \cdot x \oplus C_{Q_{i,j}}$, here x is a 64-bit variable, the linear part L_Q is a block diagonal matrix being composed of eight 8×8-bit matrices, and the constant part $C_{Q_{i,j}}$ consists of eight concatenated 8-bit vectors.

The final output of this part is the XOR of the four 64-bit outputs of the four lookup tables, which is denoted by $Y_i = Y_{i0}||Y_{i1}$ with Y_{i0} being supposed to be the original useful 32-bit value.

3.1.3. Part 3 – Implement $Y_{i0} \oplus X_i \mapsto X_{i+4}$

This part first extracts the original useful 32-bit value from the 64-bit expanded output of the second part, and then calculates X_{i+4}, as follows.

$$Y'_{i0} = P''_{i+4} \circ (Q_i)_t^{-1}(Y_{i0}),$$
$$X'_i = P'_{i+4} \circ P_i^{-1}(X_i),$$
$$X_{i+4} = Y'_{i0} \oplus X'_i,$$

where $(Q_i)_t^{-1}$ represents the corresponding part of the inverse of the encodings $L_Q \cdot x \oplus (C_{Q_{i,0}} \oplus C_{Q_{i,1}} \oplus C_{Q_{i,2}} \oplus C_{Q_{i,3}})$ of the second part, and P'_{i+4} and P''_{i+4} are new affine output encodings of the forms $P'_{i+4}(x) = P_{i+4}(x) \oplus a'_{i+4}$ and $P''_{i+4}(x) = P_{i+4}(x) \oplus a''_{i+4}$, respectively, so that X_{i+4} is a protected form with an affine output encoding $P_{i+4}(x) = A_{i+4} \cdot x \oplus a_{i+4}$, like X_i.

As a result, the whole white-box SM4 implementation can be obtained by iterating the above process for all the 32 rounds with possibly independent encodings.

Yao and Chen analysed its security against a variety of attack techniques like Billet et al.'s attack, and got that the attack complexity using affine equivalence technique was 2^{97}, and the lowest attack complexity was 2^{51} among all used attack techniques.

3.2. Security of Yao and Chen's Implementation against Collision-Based Attack

In this subsection, we analyse the security of Yao and Chen's white-box SM4 implementation against Lepoint et al.'s collision-based attack method. SM4 has a different structure and different operations from AES, and Yao and Chen's white-box SM4 implementation is distinct from Chow et al.'s white-box AES implementation: there are dummy states with indeterminate positions and the encoding used in X_{i+4} involves a general 32×32-bit matrix, which does not allow us to apply Lepoint et al.'s attack idea efficiently within one round, as for Chow et al.'s white-box AES implementation. However, we find a seemingly feasible collision function by considering two consecutive rounds for Yao and Chen's implementation, as follows.

3.2.1. Devising a Collision Function

As illustrated in Fig. 3 at a high level, the collision function used for our cryptanalysis takes as input the two 32-bit input parameters $(z_{i,0}||z_{i,1}||z_{i,2}||z_{i,3}, X_i)$ in the second part of an encryption round of Yao and Chen's implementation, and ends with the output of an $E_{i+1,j}$ operation of the X_{i+4} branch in the first part of the next encryption round ($j = 0, 1, 2, 3$). Observe that E_i and E_{i+1} are diagonal affine transformations, $E_{i,j}$ and $E_{i+1,j}$ are invertible 8×8-bit affine transformations, and $z_{i,j}$ is the original input byte to the j-th original S-box of the i-th encryption round in a protected form with $E_{i,j}$.

The collision function is functionally equivalent and can be simplified to the one depicted in Fig. 4. In our cryptanalysis and all subsequent descriptions, we set X_i such that $P'_{i+4} \circ P_i^{-1}(X_i) = 0$, and denote the constant

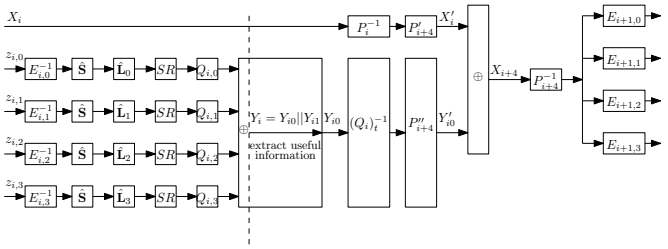

FIGURE 3. Our collision function on Yao and Chen's white-box SM4 implementation

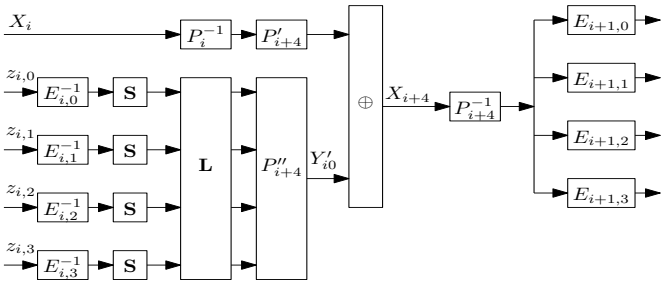

FIGURE 4. Equivalent of collision function on Yao and Chen's white-box SM4 implementation

$A_{i+4}^{-1} \cdot a''_{i+4} \oplus A_{i+4}^{-1} \circ P'_{i+4} \circ P_i^{-1}(X_i) = A_{i+4}^{-1} \cdot a''_{i+4}$ by ε_i. We now explain where the value ε_i comes from. Let $\widehat{Y}_i$ denotes the original 32-bit value immediately after the $\mathbf{L}$ operation under the input $Z_i = (z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3})$, then we have

$$\begin{aligned}
&P_{i+4}^{-1} \circ (Y'_{i0} \oplus P'_{i+4} \circ P_i^{-1}(X_i)) \\
&= P_{i+4}^{-1}(Y'_{i0}) \oplus A_{i+4}^{-1} \circ P'_{i+4} \circ P_i^{-1}(X_i) \\
&= P_{i+4}^{-1} \circ P''_{i+4}(\widehat{Y}_i) \\
&= P_{i+4}^{-1} \circ (P_{i+4}(\widehat{Y}_i) \oplus a''_{i+4}) \\
&= P_{i+4}^{-1} \circ (A_{i+4}(\widehat{Y}_i) \oplus a_{i+4} \oplus a''_{i+4}) \\
&= A_{i+4}^{-1} \circ (A_{i+4}(\widehat{Y}_i) \oplus a_{i+4} \oplus a''_{i+4} \oplus a_{i+4}) \\
&= \widehat{Y}_i \oplus A_{i+4}^{-1} \cdot a''_{i+4},
\end{aligned}$$

which is equal to $\widehat{Y}_i \oplus \varepsilon_i$ under $P'_{i+4} \circ P_i^{-1}(X_i) = 0$.

As a consequence, the collision function denoted by $f^i(z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3}, X_i)$, or simply $f^i(z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3})$ under $P'_{i+4} \circ P_i^{-1}(X_i) = 0$, is

$$\begin{aligned}
&f^i(z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3}) \\
&= \begin{bmatrix} E_{i+1,0} \\ E_{i+1,1} \\ E_{i+1,2} \\ E_{i+1,3} \end{bmatrix} \circ \oplus_{\varepsilon_i} \circ \mathbf{L} \circ \begin{bmatrix} \mathbf{S} \circ \oplus_{rk_{i,0}} \circ E_{i,0}^{-1}(z_{i,0}) \\ \mathbf{S} \circ \oplus_{rk_{i,1}} \circ E_{i,1}^{-1}(z_{i,1}) \\ \mathbf{S} \circ \oplus_{rk_{i,2}} \circ E_{i,2}^{-1}(z_{i,2}) \\ \mathbf{S} \circ \oplus_{rk_{i,3}} \circ E_{i,3}^{-1}(z_{i,3}) \end{bmatrix},
\end{aligned}$$

where $0 \le i \le 30$.

Furthermore, we express f^i as a concatenation of four byte functions f_0^i, f_1^i, f_2^i and f_3^i:

$$\begin{aligned}
&f^i(z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3}) \\
=&[f_0^i(z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3}), f_1^i(z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3}), \\
&f_2^i(z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3}), f_3^i(z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3})]^T;
\end{aligned}$$

and define $\mathbf{S}_j^i$ function as

$$\begin{aligned}
\mathbf{S}_j^i(\cdot) &= \mathbf{S} \circ \oplus_{rk_{i,j}} \circ E_{i,j}^{-1}(\cdot) \\
&= \mathbf{S}(rk_{i,j} \oplus E_{i,j}^{-1}(\cdot)), \quad j = 0, 1, 2, 3. \qquad (4)
\end{aligned}$$

3.2.2. A Linear System on $\mathbf{S}_j^i$ Functions

Next we would like to recover the functions $\mathbf{S}_0^i$, $\mathbf{S}_1^i$, $\mathbf{S}_2^i$ and $\mathbf{S}_3^i$ by exploiting collisions on the output of the functions f_j^i, in a way similar to Lepoint et al.'s. We first use the following collision to recover $\mathbf{S}_0^i$ and $\mathbf{S}_1^i$:

$$f_0^i(\alpha, 0, 0, 0) = f_0^i(0, \beta, 0, 0), \qquad (5)$$

where $\alpha, \beta \in \mathrm{GF}(2)^8$. By the linear transformation $\mathbf{L}$ in Eq. (3), Eq. (5) immediately means the following equation:

$$\begin{aligned}
&E_{i+1,0} \circ \oplus_{\varepsilon_{i,0}} \circ \big(B_1 \circ \mathbf{S}_0^i(\alpha) \oplus B_2 \circ \mathbf{S}_1^i(0) \oplus \\
&B_2 \circ \mathbf{S}_2^i(0) \oplus B_3 \circ \mathbf{S}_3^i(0)\big) \\
= \quad &E_{i+1,0} \circ \oplus_{\varepsilon_{i,0}} \circ \big(B_1 \circ \mathbf{S}_0^i(0) \oplus B_2 \circ \mathbf{S}_1^i(\beta) \oplus \\
&B_2 \circ \mathbf{S}_2^i(0) \oplus B_3 \circ \mathbf{S}_3^i(0)\big),
\end{aligned}$$

where $\varepsilon_{i,0}$ is the corresponding byte of the constant ε_i. Since $E_{i+1,0}$ is a bijection, we have the following equation:

$$B_1 \circ \mathbf{S}_0^i(\alpha) \oplus B_2 \circ \mathbf{S}_1^i(0) = B_1 \circ \mathbf{S}_0^i(0) \oplus B_2 \circ \mathbf{S}_1^i(\beta).$$

For convenience, define $u_m = \mathbf{S}_0^i(m)$ and $v_m = \mathbf{S}_1^i(m)$, then we have

$$B_1 \circ (u_0 \oplus u_\alpha) = B_2 \circ (v_0 \oplus v_\beta). \qquad (6)$$

Since $\alpha \mapsto f_0^i(\alpha, 0, 0, 0)$ and $\beta \mapsto f_0^i(0, \beta, 0, 0)$ are bijections, we can find 256 collisions. After removing $(\alpha, \beta) = (0, 0)$, we get 255 pairs (α, β) satisfying Eq. (5), each providing an equation of the form of Eq. (6). In the same way, we use other f_j^i functions ($j \in \{1, 2, 3\}$) to generate similar equations with different coefficients in $\{B_1, B_2, B_3\}$. Finally, we get 4×255 linear equations with all 512 unknowns, as follows:

$$\begin{cases}
B_1 \circ (u_0 \oplus u_\alpha) = B_2 \circ (v_0 \oplus v_{\beta_0}), \\
B_3 \circ (u_0 \oplus u_\alpha) = B_1 \circ (v_0 \oplus v_{\beta_1}), \\
B_2 \circ (u_0 \oplus u_\alpha) = B_3 \circ (v_0 \oplus v_{\beta_2}), \\
B_2 \circ (u_0 \oplus u_\alpha) = B_2 \circ (v_0 \oplus v_{\beta_3}),
\end{cases} \qquad (7)$$

where $\beta_0, \beta_1, \beta_2, \beta_3 \in [1, 255]$.

Define $u'_m = u_0 \oplus u_m$ and $v'_m = v_0 \oplus v_m$, with $m \in \{1, 2, \cdots, 255\}$, so that the number of unknowns is reduced to $2 \times 255 = 510$. Thus, Eq. (6) can be rewritten as

$$B_1 \circ u'_\alpha = B_2 \circ v'_\beta,$$

meaning that the linear system of Eq. (7) can be represented with 510 unknowns as

$$\begin{cases} B_1 \circ u'_\alpha = B_2 \circ v'_{\beta_0}, \\ B_3 \circ u'_\alpha = B_1 \circ v'_{\beta_1}, \\ B_2 \circ u'_\alpha = B_3 \circ v'_{\beta_2}, \\ B_2 \circ u'_\alpha = B_2 \circ v'_{\beta_3}. \end{cases} \tag{8}$$

3.2.3. A Small-Scale Experimental Result

We have experimentally tested the ranks of the linear system (8) under one hundred million different (affine encodings, round key) combinations (specifically, ten thousand sets of affine encodings under each of ten thousand round keys), and the rank is always 493, which is much less than the number 510 of unknowns, meaning that there are too many solutions for (u'_m, v'_m). Consequently, it is not efficient to recover (u'_m, v'_m) by this way. Thus, this small-scale experiment shows that Lepoint et al.'s collision-based attack method cannot apply to Yao and Chen's implementation. But nevertheless, the number of tested affine encodings is very small compared with the whole space, it is not possible to experimentally test all encodings, and we are not clear about whether there exist affine encodings such that the rank of the corresponding linear system is slightly less than the number of the involved unknowns. We leave it as an open problem to theoretically investigate the distribution of the ranks of the linear system (8) under all encodings.

3.2.4. A Supposed Case

Our above-mentioned experiment only tests a very small fraction of all possible encodings, and we are not clear about whether there exist affine encodings such that the rank of the corresponding linear system is slightly less than the number of the involved unknowns. Below we suppose there exist affine encodings such that the resulting linear system (8) has a rank of 509. In such a linear equation system, all other unknowns can be expressed as a function of one of them, say u'_1, that is, there exist coefficients a_i and b_i such that $u'_m = a_m \cdot u'_1$ and $v'_m = b_m \cdot u'_1$. That is,

$$\begin{aligned} u_m &= a_m \cdot (u_0 \oplus u_1) \oplus u_0, \\ v_m &= b_m \cdot (u_0 \oplus u_1) \oplus v_0. \end{aligned} \tag{9}$$

Next we can recover the $\mathbf{S}_0^i$ function by exhaustive search on the pair (u_0, u_1), and further verify whether the obtained $\mathbf{S}_0^i$ function is right or not by checking the degree of the following equation obtained from the definition of the $\mathbf{S}_0^i$ function:

$$\mathbf{S}^{-1} \circ \mathbf{S}_0^i(\cdot) = rk_{i,0} \oplus E_{i,0}^{-1}(\cdot).$$

Since $E_{i,0}^{-1}$ is an 8×8-bit invertible affine transformation, the above function has an algebraic degree of at most 1. For a wrong pair (u_0, u_1), a wrong candidate function $\widehat{\mathbf{S}}_0^i$ would be got which is an affine equivalent to $\mathbf{S}_0^i$, namely there exists an 8×8-bit matrix a and an 8-bit vector b such that $\widehat{\mathbf{S}}_0^i(\cdot) = a \cdot \mathbf{S}_0^i(\cdot) \oplus b$, with $a \neq 0$ and $(a, b) \neq (0, 1)$. The function $\mathbf{S}^{-1} \circ \widehat{\mathbf{S}}_0^i(\cdot)$ satisfies

$$\mathbf{S}^{-1} \circ \widehat{\mathbf{S}}_0^i(\cdot) = \mathbf{S}^{-1}\big(a \cdot \mathbf{S}\big(rk_{i,0} \oplus E_{i,0}^{-1}(\cdot)\big) \oplus b\big),$$

and it has an algebraic degree greater than 1 with an overwhelming probability. More specifically, we set the function $\hat{g}(\cdot) = \mathbf{S}^{-1} \circ \widehat{\mathbf{S}}^i(\cdot)$, used Lai's higher-order derivative concept [32] to calculate the first-order derivative of $\hat{g}$, and finally ran ten thousand tests without obtaining a function with an algebraic degree of 1 or less. For instance, the first-order derivative $\hat{\varphi}$ at position (`0x01`) is set to

$$\hat{\varphi}_{0x01}(x) = \hat{g}(x \oplus \texttt{0x01}) \oplus \hat{g}(x),$$

and we verify whether $\hat{\varphi}_{0x01}(x)$ is constant with at most 2^7 inputs of x, since $\hat{\varphi}_{0x01}(x) = \hat{\varphi}_{0x01}(x \oplus \texttt{0x01})$. For each wrong pair, the probability that $\hat{\varphi}(x)$ is constant is roughly 2^{-8}, so wrong guesses can be quickly removed.

After recovering $\mathbf{S}_0^i$, we can use Eq. (9) to recover $\mathbf{S}_1^i$ by exhaustive search on v_0, and similarly recover $\mathbf{S}_2^i$ and $\mathbf{S}_3^i$ with other equations finally.

3.2.4.1 Recovering Encoding P''_{i+4} and Linear Parts of Encodings P_{i+4} and P'_{i+4}

After recovering $\mathbf{S}_0^i$, $\mathbf{S}_1^i$, $\mathbf{S}_2^i$ and $\mathbf{S}_3^i$ functions in the first phase, we can easily recover the 32×32-bit affine encoding P''_{i+4} used in the i-th round, see Fig. 2 or Fig. 4, since $Y'_{i0} = P''_{i+4}(\mathbf{L} \circ (\mathbf{S}_0^i(z_{i,0})||\mathbf{S}_1^i(z_{i,1})||\mathbf{S}_2^i(z_{i,2})||\mathbf{S}_3^i(z_{i,3})))$. Recall that $P_i(x) = A_i \cdot x \oplus a_i$, $P'_{i+4}(x) = P_{i+4} \oplus a'_{i+4}$ and $P''_{i+4}(x) = P_{i+4} \oplus a''_{i+4}$, we can learn that $a_{i+4} = a'_{i+4} \oplus a''_{i+4}$, and consequently we can recover both the linear part A_{i+4} and the constant part a'_{i+4} of P''_{i+4}, as follows. We recover the linear part A_{i+4} in a way similar to recovering the linear part of encoding E_{i+1} above, that is, by considering the output difference under a pair of inputs:

$$\begin{aligned} & Y'_{i0} \oplus \hat{Y}'_{i0} \\ = \; & P''_{i+4}(\mathbf{L} \circ (\mathbf{S}_0^i(z_{i,0})||\mathbf{S}_1^i(z_{i,1})||\mathbf{S}_2^i(z_{i,2})||\mathbf{S}_3^i(z_{i,3}))) \oplus \\ & P''_{i+4}(\mathbf{L} \circ (\mathbf{S}_0^i(\hat{z}_{i,0})||\mathbf{S}_1^i(\hat{z}_{i,1})||\mathbf{S}_2^i(\hat{z}_{i,2})||\mathbf{S}_3^i(\hat{z}_{i,3}))) \\ = \; & A_{i+4}(\mathbf{L} \circ (\mathbf{S}_0^i(z_{i,0})||\mathbf{S}_1^i(z_{i,1})||\mathbf{S}_2^i(z_{i,2})||\mathbf{S}_3^i(z_{i,3})) \oplus \\ & \mathbf{L} \circ (\mathbf{S}_0^i(\hat{z}_{i,0})||\mathbf{S}_1^i(\hat{z}_{i,1})||\mathbf{S}_2^i(\hat{z}_{i,2})||\mathbf{S}_3^i(\hat{z}_{i,3}))). \end{aligned}$$

The constant part a'_{i+4} can be easily obtained, as we have $Y'_{i0} = P''_{i+4}(\mathbf{L} \circ (\mathbf{S}_0^i(z_{i,0})||\mathbf{S}_1^i(z_{i,1})||\mathbf{S}_2^i(z_{i,2})||\mathbf{S}_3^i(z_{i,3}))) = A_{i+4}(\mathbf{L} \circ (\mathbf{S}_0^i(z_{i,0})||\mathbf{S}_1^i(z_{i,1})||\mathbf{S}_2^i(z_{i,2})||\mathbf{S}_3^i(z_{i,3}))) \oplus a'_{i+4}$.

As P_{i+4} and P'_{i+4} use the same linear part A_{i+4} as P''_{i+4}, we can know the linear part A_{i+4} of P_{i+4} or P'_{i+4}. (But it seems impossible to recover a_{i+4} and a''_{i+4} subsequently.)

3.2.4.2 Recovering the Linear Part of Encoding E_{i+1}

After the $\mathbf{S}_j^i$ functions ($i = 0, 1, \cdots, 30$) have been recovered ($j = 0, 1, 2, 3$), however it is not possible to

recover the output encodings $E_{i+1,j}$ as Lepoint et al.'s attack on Chow et al.'s white-box AES implementation, because of the existence of the unknown constant ε_i, which is partially due to the different structures of Feistel and SPN ciphers and the design of Yao and Chen's implementation. Anyway, we can recover the linear part C_{i+1} of the output encoding E_{i+1} by considering the output difference under a pair of inputs to the f_j^i functions, as follows.

Given a 32-bit input $Z_i = (z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3})$ to the f^i collision function, denote the original 32-bit value immediately after the $\widehat{\mathbf{L}}$ operation as follows:

$$\begin{aligned} W_i &= \begin{bmatrix} W_{i,0} & W_{i,1} & W_{i,2} & W_{i,3} \end{bmatrix}^T \\ &= \mathbf{L}_0 \circ \mathbf{S}_0^i(z_{i,0}) \oplus \mathbf{L}_1 \circ \mathbf{S}_1^i(z_{i,1}) \oplus \\ &\quad \mathbf{L}_2 \circ \mathbf{S}_2^i(z_{i,2}) \oplus \mathbf{L}_3 \circ \mathbf{S}_3^i(z_{i,3}). \end{aligned}$$

As $\mathbf{L}$ is public and we have recovered $\mathbf{S}_j^i$ above ($j = 0, 1, 2, 3$), we can compute Y_j. The output of the f^i collision function is

$$f^i = \begin{bmatrix} f_0^i & f_1^i & f_2^i & f_3^i \end{bmatrix}^T = \begin{bmatrix} E_{i+1,0}(W_{i,0} \oplus \varepsilon_{i,0}) \\ E_{i+1,1}(W_{i,1} \oplus \varepsilon_{i,1}) \\ E_{i+1,2}(W_{i,2} \oplus \varepsilon_{i,2}) \\ E_{i+1,3}(W_{i,3} \oplus \varepsilon_{i,3}) \end{bmatrix},$$

where $(\varepsilon_{i,0}, \varepsilon_{i,1}, \varepsilon_{i,2}, \varepsilon_{i,3}) = \varepsilon_i$.

Subsequently, to recover $E_{i+1,j}$, we need to know the 8-bit unknown constant $\varepsilon_{i,j}$. A straightforward way is to try by exhaustive search, which would cause an additional complexity factor of 2^8. However, we find the linear part $C_{i+1,j}$ can be recovered at ease with a negligible time complexity, as follows.

First, we consider the output of the arbitrary 32-bit input $Z_i = (z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3})$ under the f_0^i collision function,

$$\begin{aligned} f_0^i(Z_i) &= E_{i+1,0}(W_{i,0} \oplus \varepsilon_{i,0}) \\ &= C_{i+1,0}(W_{i,0}) \oplus C_{i+1,0}(\varepsilon_{i,0}) \oplus c_{i+1,0}, \end{aligned} \quad (10)$$

where $W_{i,0}$ is defined above, which denotes the corresponding original 8-bit value immediately after the $\mathbf{L}$ operation under the input X.

Next, we choose the 32-bit input $Z0 = (\hat{z}_{i,0}, \hat{z}_{i,1}, \hat{z}_{i,2}, \hat{z}_{i,3})$ to the f^i collision function, so that the original 32-bit value immediately after the $\mathbf{L}$ operation is 0; this can be done easily by choosing $Z0$ such that

$$(\mathbf{S}_0^i(\hat{z}_{i,0}), \mathbf{S}_1^i(\hat{z}_{i,1}), \mathbf{S}_2^i(\hat{z}_{i,2}), \mathbf{S}_3^i(\hat{z}_{i,3})) = \mathbf{L}^{-1}(0) = 0.$$

Thus, its corresponding output under the f_0^i collision function is

$$f_0^i(X0) = C_{i+1,0}(\varepsilon_{i,0}) \oplus c_{i+1,0}. \quad (11)$$

At last, XORing Eq. (10) and Eq. (11), we get $f_0^i(Z_i) \oplus f_0^i(Z0) = C_{i+1,0}(W_{i,0})$. As a consequence, we can recover the linear part $C_{i+1,0}$ of the output encodings $E_{i+1,0}$. The linear parts of other output encodings $E_{i+1,j}$ can be recovered similarly.

3.2.4.3 Recovering Masked Round Key $rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1})$

We similarly define the collision function f^{i+1} starting from the $(i+1)$-th round ($0 \le i \le 30$). Subsequently, as Lepoint et al. did on Chow et al.'s white-box AES implementation, we would like to recover the round key rk_{i+1} in the following $(i+1)$-th round, but nevertheless we cannot recover a round key byte from the collision function, because $\varepsilon_{i+1,0}$ and $E_{i+1,j}$ are unknown, although the linear part $C_{i+1,j}$ can be recovered as above. At present we can only recover the masked round key $rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1})$, as follows.

Building on some equations similar to Eq. (4) and Eq. (5), we define

$$\begin{aligned} g(x) &= f_0^{i+1}\big(E_{i+1,0}(\mathbf{S}^{-1}(x) \oplus rk_{i+1,0}), 0, 0, 0\big) \\ &= f_0^{i+1}\big(C_{i+1,0}(\mathbf{S}^{-1}(x)) \oplus E_{i+1,0}(rk_{i+1,0}), 0, 0, 0\big) \\ &= E_{i+2,0}(B_1(x) \oplus \delta \oplus \varepsilon_{i+1,0}), \end{aligned}$$

where $\delta = B_2 \circ \mathbf{S}_1^{i+1}(0) \oplus B_2 \circ \mathbf{S}_2^{i+1}(0) \oplus B_3 \circ \mathbf{S}_3^{i+1}(0)$ is a constant that can be easily computed.

Because $E_{i+2,0}$ is an 8×8-bit affine transformation, the function g has an algebraic degree of at most 1. For a wrong guess $E_{i+1,0}(\hat{rk}_{i+1,0}) \neq E_{i+1,0}(rk_{i+1,0})$, the function $\hat{g}$ is defined as

$$\begin{aligned} \hat{g}(x) &= f_0^{i+1}\big(E_{i+1,0}(\mathbf{S}^{-1}(x) \oplus \hat{rk}_{i+1,0}), 0, 0, 0\big) \\ &= f_0^{i+1}\big(C_{i+1,0}(\mathbf{S}^{-1}(x)) \oplus E_{i+1,0}(\hat{rk}_{i+1,0}), 0, 0, 0\big) \\ &= E_{i+2,0}(B_1 \circ \mathbf{S}(\mathbf{S}^{-1}(x) \oplus \hat{rk}_{i+1,0} \oplus rk_{i+1,0}) \oplus \\ &\quad \delta \oplus \varepsilon_{i+1,0}). \end{aligned}$$

In this case, with a similar test, $\hat{g}$ has an algebraic degree of more than 1 with an overwhelming probability. We extract $E_{i+1,0}(rk_{i+1,0})$ by exhaustive search, that is, similarly we verify whether the first-order derivative $\hat{\varphi}(x) = \hat{g}(x \oplus \texttt{0x01}) \oplus \hat{g}(x)$ of $\hat{g}(x)$ at point $\texttt{0x01}$ is constant for each guess $E_{i+1,0}(\hat{rk}_{i+1,0})$. For a wrong guess $E_{i+1,0}(\hat{rk}_{i+1,0})$, the probability that $\hat{\varphi}(x)$ is constant is roughly 2^{-8}, so wrong guesses can be quickly removed.

Similarly, we can recover $E_{i+1,j}(rk_{i+1,j})$ for $j = 1, 2, 3$, by changing the definition of the function g. Since $E_{i+1}(rk_{i+1}) = C_{i+1}(rk_{i+1}) \oplus c_{i+1} = C_{i+1}(rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1}))$, we can get the masked round key $rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1})$, where $C_{i+1}^{-1}(c_{i+1})$ is unknown.

3.2.4.4 Recovering the Linear Part of Encoding E_0 and Masked Round Key $rk_0 \oplus C_0^{-1}(c_0)$

After recovering $\mathbf{S}_j^0$, we can recover the linear part $C_{0,j}$ of the encoding $E_{0,j}$ by considering the output difference under a pair of inputs to the $\mathbf{S}_j^0$ function, say $(x, y = \mathbf{S}_j^0(x) = \mathbf{S}(rk_{0,j} \oplus E_{0,j}^{-1}(x)))$ and $(x', y' = \mathbf{S}_j^0(x') = \mathbf{S}(rk_{0,j} \oplus E_{0,j}^{-1}(x')))$, then we have $C_{0,j}^{-1}(x \oplus x') = \mathbf{S}^{-1}(y) \oplus \mathbf{S}^{-1}(y')$, and thus we can recover $C_{0,j}$. Further, we can recover the masked round key $rk_0 \oplus C_0^{-1}(c_0)$ as $rk_{0,j} \oplus c_{0,j} = \mathbf{S}^{-1}(y) \oplus C_{0,j}^{-1}(x)$.

Notice that this way can also recover the linear part $C_{i+1,j}$ of the encoding $E_{i+1,j}$ and the masked round key $rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1})$ after the $\mathbf{S}_j^{i+1}$ functions are recovered as recovering the $\mathbf{S}_j^i$ functions before ($i = 0, 1, \cdots, 30$, $j = 0, 1, 2, 3$), but it is relatively costly when we target to recover them for all the first 31 rounds.

3.2.4.5 Attack Complexity

In the phase of recovering $\mathbf{S}_0^i$, there are 2^{16} candidates (u_0, u_1) for exhaustive search, and we need to calculate $\hat{\varphi}(x)$ for at most 2^7 inputs to verify whether $\hat{\varphi}(x)$ is constant, where the probability that $\hat{\varphi}(x)$ is constant is roughly 2^{-8} for a wrong guess (u_0, u_1), and thus the expected value of the test is $1 + \frac{1}{256} + \cdots + (\frac{1}{256})^{127} \approx 1$. So the expected time complexity of recovering $\mathbf{S}_0^i$ is hence about $2^{16} \cdot 2 \cdot 1 = 2^{17}$ $\mathbf{S}$ (or $\mathbf{S}^{-1}$) computations, where 2^{16} represents the number of candidates (u_0, u_1), "2" represents two $\mathbf{S}^{-1}$ computations for every verification $\hat{\varphi}(x)$, "1" represents the expected value of the test on $\hat{\varphi}(x)$, and the time complexity for solving the linear system of 4×255 equations is negligible because it is very sparse. We recover $\mathbf{S}_1^i$, $\mathbf{S}_2^i$ and $\mathbf{S}_3^i$ by exhaustive search on v_0 and produce an expected time complexity of $3 \cdot (2^8 \cdot 1 \cdot 2) = 3 \cdot 2^9$ $\mathbf{S}$ computations. Thus, the expected time complexity of recovering all the four $\mathbf{S}_j^i$'s is about $2^{17} + 3 \cdot 2^9 = 259 \cdot 2^9$ $\mathbf{S}$ computations. The time complexity for recovering the linear part of output encoding $E_{i+1,j}$ is negligible. The expected time complexity of recovering $rk_{i+1,0} \oplus C_{i+1,0}^{-1}(c_{i+1,0})$ is about $2^8 \cdot 1 \cdot 2 = 2^9$ $\mathbf{S}$ computations, so the total expected time complexity of recovering $E_{i+1}(rk_{i+1})$ (or $rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1})$) is about $259 \cdot 2^9 + 4 \cdot (2^8 \cdot 1 \cdot 2) = 259 \cdot 2^9 + 2^{11}$ $\mathbf{S}$ computations. After we recover $rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1})$, it is easy to recover P''_{i+5} and the linear parts of P'_{i+5} and P_{i+5} used in the same round.

Therefore, in case there exist such affine encodings that make the resulting linear system (8) have a rank of 509, Lepoint et al.'s collision-based attack method can be applied to remove most white-box operations until mainly some Boolean masks remain and recover the masked round keys $rk_i \oplus C_i^{-1}(c_i)$ in the first 31 rounds with an expected time complexity of about $259 \cdot 2^9 + 30 \times 2^{11} \approx 2^{17.4}$ $\mathbf{S}$ computations, that is to say, Yao and Chen's implementation can be somewhat equivalently simplified to an implementation with mainly Boolean masks in this case.

4. SECURITY OF XIAO AND LAI'S WHITE-BOX SM4 IMPLEMENTATION AGAINST COLLISION-BASED ATTACK

Xiao and Lai's white-box SM4 implementation [19] is similar to Yao and Chen's white-box SM4 implementation at a high level, except that there is no state expansion to the S-box layer and thus the original $\mathbf{L}$ operation is used. Therefore, our above cryptanalysis on Yao and Chen's implementation can be applied to Xiao and Lai's implementation. That is, a small-scale experiment shows that the rank of the concerned linear system is much less than the number of the involved unknowns, meaning that Lepoint et al.'s collision-based attack method cannot apply to Xiao and Lai's implementation, but it is an open problem whether there exist affine encodings such that the corresponding linear system has a rank of 509, in which case Lepoint et al.'s collision-based attack method can be applied to remove most white-box operations until mainly some Boolean masks remain and recover masked round keys in the first 31 rounds with an expected time complexity of about $2^{17.4}$ $\mathbf{S}$ computations.

5. SECURITY OF SHI ET AL.'S WHITE-BOX SM4 IMPLEMENTATION AGAINST COLLISION-BASED ATTACK

Shi et al.'s white-box SM4 implementation [22] is based on the idea of dual cipher [23], but as shown by Wang et al. [30,31], it can be transformed to the construction of Fig. 4 with the affine encodings being replaced with linear encodings. Thus, our above cryptanalysis on Yao and Chen's implementation can be similarly applied to Shi et al.'s implementation, a small-scale experiment shows that the rank of the concerned linear system is much less than the number of the involved unknowns, meaning that Lepoint et al.'s collision-based attack method cannot apply to Shi et al.'s implementation, but it is an open problem whether there exist affine encodings such that the corresponding linear system has a rank of 509, in which case Lepoint et al.'s collision-based attack method can be applied to recover most white-box operations and recover a round key with an expected time complexity of about $2^{17.1}$ $\mathbf{S}$ computations, as Wang et al. did in [31] (note that Wang et al.'s time complexity 2^{23} is obtained for checking the first-order derivatives for all 2^7 inputs in the worst case, but its expected time complexity is around $2^{17.1}$).

6. SECURITY OF SHANG'S WHITE-BOX SM4 IMPLEMENTATION AGAINST COLLISION-BASED ATTACK

Shang's white-box SM4 implementation [24] is based on Xiao and Lai's white-box SM4 implementation, mainly by applying two general 16-bit affine encodings $E_{i,0}$ and $E_{i,1}$ to the input of the S-box layer in parallel, each corresponding to two S-boxes and subsequently a 32×16-bit component $\mathbf{L}_0$ or $\mathbf{L}_1$ of the $\mathbf{L}$ matrix, and thus two 16×32-bit tables. Fig. 5 depicts an encryption round of Shang's implementation.

We would like to apply Lepoint et al.'s collision-based attack method to Shang's implementation. Specifically, we represent $\mathbf{L}$ by Eq. (3) with two 16×16-bit blocks $\hat{L}_0$ and $\hat{L}_1$ as

$$\mathbf{L} = \begin{bmatrix} \hat{L}_0 & \hat{L}_1 \\ \hat{L}_1 & \hat{L}_0 \end{bmatrix},$$

define

$$\mathbf{S}_0^i(x) = \begin{pmatrix}\mathbf{S}\\ \mathbf{S}\end{pmatrix}\big((rk_{i,0}||rk_{i,1}) \oplus E_{i,0}(x)\big),$$
$$\mathbf{S}_1^i(x) = \begin{pmatrix}\mathbf{S}\\ \mathbf{S}\end{pmatrix}\big((rk_{i,2}||rk_{i,3}) \oplus E_{i,1}(x)\big),$$

where $x \in \mathrm{GF}(2)^{16}$, and define a collision function on Shang's implementation whose equivalent f^i is depicted in Fig. 6, as follows:

$$\begin{aligned} &f^i(z_{i,0}, z_{i,1}) = [f_0^i(z_{i,0}, z_{i,1}), f_1^i(z_{i,0}, z_{i,1})]^T \\ &= \begin{bmatrix}E_{i+1,0}^{-1}\\ E_{i+1,1}^{-1}\end{bmatrix} \oplus_{\epsilon_i} \circ\mathbf{L} \circ \begin{bmatrix}\begin{pmatrix}\mathbf{S}\\ \mathbf{S}\end{pmatrix}\big((rk_{i,0}||rk_{i,1}) \oplus E_{i,0}(z_{i,0})\big)\\ \begin{pmatrix}\mathbf{S}\\ \mathbf{S}\end{pmatrix}\big((rk_{i,2}||rk_{i,3}) \oplus E_{i,1}(z_{i,1})\big)\end{bmatrix} \\ &= \begin{bmatrix}E_{i+1,0}^{-1}\\ E_{i+1,1}^{-1}\end{bmatrix} \oplus_{\epsilon_i} \circ\mathbf{L} \circ \begin{bmatrix}\mathbf{S}_0^i(z_{i,0})\\ \mathbf{S}_1^i(z_{i,1})\end{bmatrix}. \end{aligned}$$

Then, we consider the collision $f_h^i(\alpha, 0) = f_h^i(0, \beta)$, where $\alpha, \beta \in \mathrm{GF}(2)^{16}$ and $h = 0, 1$. At last, we get the following linear system of $2 \times (2^{16} - 1)$ equations with $2 \times (2^{16} - 1)$ unknowns:

$$\begin{aligned} \hat{L}_0 \circ u'_\alpha &= \hat{L}_1 \circ v'_{\beta_0},\\ \hat{L}_1 \circ u'_\alpha &= \hat{L}_0 \circ v'_{\beta_1}, \end{aligned} \tag{12}$$

where $u'_\alpha = \mathbf{S}_0^i(\alpha) \oplus \mathbf{S}_0^i(0)$ and $v'_{\beta_h} = \mathbf{S}_1^i(\beta_h) \oplus \mathbf{S}_1^i(0)$ and $(\alpha, \beta_h) \neq (0, 0)$.

6.1. A Small-Scale Experimental Result

The size of the linear system made up of Eqs. (12) is very large, and an experimental test on the rank of the linear system under a single (affine encodings, round key) combination takes about two and a half hours on a workstation (Intel(R)Xeon(R) Platinum 8280 CPU @ 270GHz(56 CPUs), 2.7GHz), which is very time-consuming. We have experimentally tested 15 different (affine encodings, round key) combinations, but the rank is always 129785, which is much less than the number 131070 of unknowns, meaning that there are too many solutions for (u'_α, v'_β). Consequently, it is not efficient to recover (u'_α, v'_β) by this way. Thus, our experimental result means that Lepoint et al.'s collision-based attack method cannot apply to Shang's implementation. But again, the number of tested encodings is very small compared with the whole space, and it is an open problem whether there exist affine encodings such that the rank of the corresponding linear system is slightly less than the number of the involved unknowns.

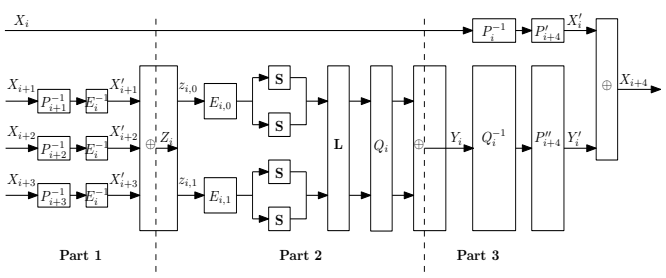

FIGURE 5. An encryption round of Shang's white-box SM4 implementation

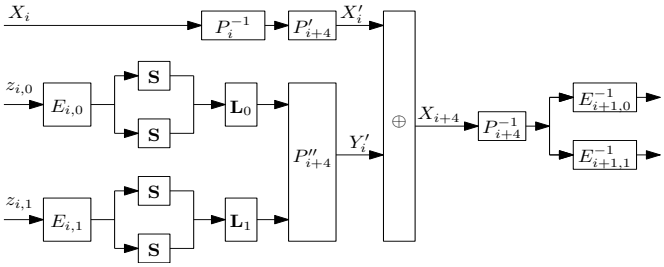

FIGURE 6. Equivalent of collision function on Shang's white-box SM4 implementation

6.2. A Supposed Case

Suppose there exist affine encodings such that the resulting linear system (12) has a rank of $2 \times (2^{16} - 1) - 1 = 131069$. Thus, by a similar process, we can recover the $\mathbf{S}_h^i$ function with an expected time complexity of about $2^{32} \cdot 1 \cdot 2 \cdot 2 + 2^{16} \cdot 1 \cdot 2 \cdot 2 = 2^{34} + 2^{18}$ **S** computations, and recover the linear part of $E_{i+1,h}$ with a negligible time complexity. Note that each $\mathbf{S}_h^i$ computation involves two **S** computations here.

At last, suppose that $E_{i+1,h}(\cdot) = C_{i+1,h}(\cdot) \oplus c_{i+1,h}$ and the linear part $C_{i+1,h}$ has been recovered as above, where $C_{i+1,h}$ is a general invertible 16×16-bit matrix and $c_{i+1,h}$ is a 16-bit constant. Similarly, we depend on the following function to recover the masked key bytes $(rk_{i+1,0}||rk_{i+1,1}) \oplus C_{i+1,h}^{-1}(c_{i+1,h})$ with an expected time complexity of about $2^{16} \cdot 1 \cdot 2 \cdot 2 = 2^{18}$ **S** computations:

$$\begin{aligned} &f_0^{i+1}\big(E_{i+1,0}^{-1}\big(\begin{pmatrix}\mathbf{S}^{-1}\\ \mathbf{S}^{-1}\end{pmatrix}(x) \oplus (rk_{i+1,0}||rk_{i+1,1})\big), 0\big)\\ &= f_0^{i+1}\big(C_{i+1,0}^{-1}\big(\begin{pmatrix}\mathbf{S}^{-1}\\ \mathbf{S}^{-1}\end{pmatrix}(x)\big) \oplus E_{i+1,0}^{-1}(rk_{i+1,0}||rk_{i+1,1}), 0\big)\\ &= E_{i+2,0}^{-1}(L_0(x) \oplus L_1 \circ \mathbf{S}_1^{i+1}(0) \oplus \varepsilon_{i+1,0}), \end{aligned}$$

where $\varepsilon_{i+1,0}$ is the corresponding 16-bit part of ε_{i+1}.

Thus, in case there exist such affine encodings that make the resulting linear system (12) have a rank of 131069, the total expected time complexity is about $2^{34} + 2^{18} + 2 \cdot 2^{18} \approx 2^{34}$ **S** computations for recovering the masked round key $rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1})$ from Shang's implementation ($i = 0, 1, \cdots, 29$), and the expected time complexity is about $2^{34} + 2^{18} + 30 \cdot 2 \cdot 2^{18} \approx 2^{34}$ **S** computations to recover all the masked round keys $rk_0 \oplus C_0^{-1}(c_0)$ and $rk_{i+1} \oplus C_{i+1}^{-1}(c_{i+1})$ in the first 31 rounds; in other words, we can peel off most white-box operations and make a somewhat equivalent white-box SM4 implementation mainly with Boolean masks.

7. CONCLUDING REMARKS

The SM4 block cipher is a Chinese national standard and an ISO international standard. In this paper, we have analysed the security of a type of white-box SM4 implementations that uses a linear or affine diagonal block encoding to protect the three branches entering a round function and uses its inverse as the input encoding to the S-box layer. Our small-scale experiments show that these white-box SM4 implementations can resist Lepoint et al.'s collision-based attack method, but nevertheless we leave it as an open problem whether there exist such encodings that the rank of the corresponding linear system is slightly less than the number of the involved unknowns, in which case Lepoint et al.'s collision-based attack method can be applied with a practical time complexity to recover a round key or to remove most white-box operations until some Boolean masks remain. Our cryptanalysis on this type of white-box SM4 implementations suggests to some extent that building a white-box table with one or two S-boxes is both okay and affine encodings are preferable to linear encodings in the sense of their security against Lepoint et al.'s collision-based attack method. A possible future research topic on white-box SM4 implementation is to investigate the distribution of the ranks under all encodings and learn whether they produce the same rank. By contrast, Lu et al.'s work [15] on white-box AES suggests that (for white-box AES implementation) building a white-box table with two S-boxes is better than building a white-box table with one S-box in the sense of their security against collision-based attacks, so white-box SM4 is different from white-box AES at this point.

ACKNOWLEDGEMENTS

The authors are very grateful to the anonymous reviewers for their comments, and to Prof. Zheng Gong for discussions on an earlier version of this paper. This work was supported by National Natural Science Foundation of China (No. 61972018) and Guangxi Key Laboratory of Cryptography and Information Security (No. GCIS202102). An earlier version of part materials of this paper appeared in Proceedings of ISC 2021 [33], and this is a corrected and extended version of [33]. Jiqiang Lu was Qianjiang Special Expert of Hangzhou.

DATA AVAILABILITY STATEMENT

The data underlying this article will be shared on reasonable request to the corresponding author.

REFERENCES

[1] Chow, S., Eisen, P., Johnson, H. and Van Oorschot, P. C. (2003) White-box cryptography and an AES implementation. Proceedings of SAC 2002, Newfoundland, Canada, 15-16 August, pp. 250-270. Springer, Berlin.

[2] FIPS PUB 197 (2001) Specification for the Advanced Encryption Standard (AES). National Institute of Standards and Technology, USA.

[3] Billet, O., Gilbert, H. and Ech-Chatbi, C. (2004) Cryptanalysis of a white box AES implementation. Proceedings of SAC 2004, Waterloo, Canada, 9-10 August, pp. 227-240. Springer, Berlin.

[4] Tolhuizen, L. (2012) Improved cryptanalysis of an AES implementation. Proceedings of The 33rd WIC Symposium on Information Theory in the Benelux, Boekelo, The Netherlands, 24-25 May, pp. 68-71.

[5] Lepoint, T., Rivain, M., De Mulder, Y., Roelse, P. and Preneel, B. (2014) Two attacks on a white-box AES implementation. Proceedings of SAC 2013, Burnaby, Canada, 14-16 August, pp. 265-285. Springer, Berlin.

[6] Bringer, J., Chabanne, H. and Dottax, E. (2006) White box cryptography: another attempt. IACR Cryptology ePrint Archive, 2006, 468.

[7] Xiao, Y.Y. and Lai, X.J. (2009) A secure implementation of white-box AES. Proceedings of CSA 2009, Jeju Island, Korea, 10-12 December, pp. 1-6. IEEE.

[8] Karroumi, M. (2011) Protecting white-box AES with dual ciphers. Proceedings of ICISC 2010, Seoul, Korea, 1-3 December, pp. 278-291. Springer, Berlin.

[9] Luo, R., Lai X.J. and You, R. (2014) A new attempt of white-box AES implementation. Proceedings of SPAC 2014, Wuhan, China, 18-19 November, pp. 423-429. IEEE.

[10] Baek, C.H., Cheon, J.H. and Hong, H. (2016) White-box AES implementation revisited. Journal of Communications and Networks, 18, 273-287.

[11] Bai, K.P., Wu, C.K. and Zhang, Z.F. (2018) Protect white-box AES to resist table composition attacks. IET Information Security, 12, 305-313.

[12] De Mulder, Y., Wyseur, B. and Preneel, B. (2010) Cryptanalysis of a perturbated white-box AES implementation. Proceedings of INDOCRYPT 2010, Hyderabad, India, 12-15 December, pp. 292-310. Springer, Berlin.

[13] De Mulder, Y., Roelse, P. and Preneel, B. (2013) Cryptanalysis of the Xiao-Lai white-box AES implementation. Proceedings of SAC 2012, Windsor, Canada, 15-16 August, pp. 34-49. Springer, Berlin.

[14] Derbez, P., Fouque, P.A., Lambin, B. and Minaud, B. (2018) On recovering affine encodings in white-box implementations. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2018, 121-149.

[15] Lu, J., Wang, M., Wang, C. and Yang, C. (to appear) Collision-based attacks on white-box implementations of the AES block cipher. Proceedings of SAC 2022, Ontario, Canada, 24-26 August. Springer, Switzerland.

[16] SMS4 (2006) The SMS4 cryptographic algorithm used in WLAN products (in Chinese). Office of State Commercial Cryptography Administration, Beijing, China.

[17] GB/T 32907-2016 (2016) Information Security Technology — SM4 Block Cipher Algorithm. Standardization Administration of China, Beijing, China.

[18] ISO/IEC 18033-3:2010/AMD1:2021 (2021) Information technology — Security techniques — Encryption algorithms — Part 3: Block ciphers — Amendment

1: SM4. International Standardization of Organization, Switzerland.

[19] Xiao, Y.Y. and Lai, X.J. (2009) White-box cryptography and a SMS4 implementation. Proceedings of the 2009 Annual Conference of the Chinese Association of Cryptologic Research, Guangzhou, China, 13-16 November, pp. 24–34 (in Chinese).

[20] Lin, T.T. and Lai, X.J. (2013) Efficient attack to white-box SMS4 implementation. Journal of Software, 24, 2238-2249 (in Chinese).

[21] Biham, E. and Shamir, A. (1993) Differential cryptanalysis of the Data Encryption Standard. Springer.

[22] Shi, Y., Wei, W.J. and He, Z.J. (2015) A lightweight white-box symmetric encryption algorithm against node capture for WSNs. Sensors, 15, 11928-11952.

[23] Barkan, E. and Biham, E. (2002) In how many ways can you write Rijndael?. Proceedings of ASIACRYPT 2002, Queenstown, New Zealand, 1-5 December, pp. 160-175. Springer, Berlin.

[24] Shang, P. (2016) White-box cryptography algorithm design and implementation of SMS4. Master thesis, University of Electronic Science and Technology of China, Chengdu, China.

[25] Bai, K.P. and Wu, C.K. (2016) A secure white-box SM4 implementation. Security and Communication Networks, 9, 996-1006.

[26] Pan, W.L., Qin, T.H., Jia, Y. and Zhang, L.T. (2018) Cryptanalysis of two white-box SM4 implementations. Journal of Cryptologic Research, 2018, 651-670 (in Chinese).

[27] Lin, T.T., Yan, H.L., Lai, X.J., Zhong, Y.X. and Jia, Y. (2018) Security evaluation and improvement of a white-box SMS4 implementation based on affine equivalence algorithm. The Computer Journal, 61, 1783-1790.

[28] Biryukov, A., De Cannière, C., Braeken, A. and Preneel, B. (2003) A toolbox for cryptanalysis: linear and affine equivalence algorithms. Proceedings of EUROCRYPT 2003, Warsaw, Poland, 4-8 May, pp. 33-50. Springer, Berlin.

[29] Yao, S. and Chen, J. (2020) A new method for white-box implementation of SM4 algorithm. Journal of Cryptologic Research, 2020, 358-374 (in Chinese).

[30] Wang, R. (2021) Security analysis of lightweight white-box cryptography. Master thesis, Beihang University, Beijing, China.

[31] Wang, R., Guo, H., Lu, J. and Liu, J. (2022) Cryptanalysis of a white-box SM4 implementation based on collision attack. IET Information Security, 16, 18-27.

[32] Lai, X.J. (1994) Higher order derivatives and differential cryptanalysis. In Blahut, R., Costello, D.J., Maurer, U. and Mittelholzer, T. (eds), Communications and Cryptography: Two Sides of One Tapestry, Kluwer.

[33] Lu, J. and Li, J. (2021) Cryptanalysis of two white-box implementations of the SM4 block cipher. Proceedings of ISC 2021, Bali, Indonesia, 10-12 November, pp. 54-69. Springer, Switzerland.